

chrome_tab_organiser — v1.5 Quick Guide

v1.5 Quick Guide — Import Chrome Tabs

chrome_report_edit_v1.5.py

— load a report, pick **Add tabs from window**,
choose a Chrome window or group, press **Add tabs**

chrome_report_v1.4.py

— the report generator

Document suffix `_261005` — self-contained tools, Python standard library only.

Contents

1. Do it in four clicks
2. Where the imported tabs land
3. What you get
4. Commands
5. If something looks wrong
6. Still true from earlier versions

Import tabs from the running Chrome into any report window Date suffix: `_261005` Applies to:

`chrome_report_edit_v1.5.py`

1. Do it in four clicks

1. `python chrome_report_edit_v1.5.py --dir reports`
2. Load the report
3. Click **Add tabs from window** (in the header of the target window)
4. Pick a Chrome window or a Chrome group -> **Add tabs**

The dialog lists, live:

- **Chrome windows** — `Window 1`, `Window 2`, ... with tab and group counts,

• **active** on the one Chrome is showing;

- **Chrome tab groups** — every named group, with its tab count.

A line under the list tells you what will happen, e.g. *“18 tabs will be added, inserted at the beginning of the window.”*

Chrome keeps running. Nothing in Chrome is opened, closed or modified.

2. Where the imported tabs land

“sort groups alphabetically”	Placement
unticked (default)	at the very top of the target window, in Chrome's tab-strip order
ticked	each tab placed by its own group , so the window stays sorted

3. What you get

- **Title, URL, group name and group colour** come across as they are in Chrome.
- The **active-window star** does not.
- Tabs already in the target window are **skipped**, and the count is reported:

Added 18 tabs from Window 3 (2 already in this window).

- The duplicate check ignores letter case, a trailing slash and **#fragment**,

so `Example.com`, `example.com/` and `example.com#top` are one page.

- Import the same window again and it adds nothing.

Imported rows are ordinary rows — renumbered, editable and draggable.

4. Commands

```
python chrome_report_edit_v1.5.py [--dir PATH] [--port 8765]
                                [--host 127.0.0.1] [--no-browser]
                                [--no-chrome]

chrome_report_edit_v1.5.exe --dir reports          (no Python needed)

--no-chrome      switch the live import off entirely; no session file is read
```

5. If something looks wrong

Symptom	Answer
"no open Chrome windows found"	Chrome has no session yet on this machine, or it isn't the Default profile.
The list is a moment behind	Chrome was writing its newest session file; the complete snapshot was used. Click the button again shortly.
"(--no-chrome)" in the message	Expected — you started the editor with that flag.
Window names don't match the report	Normal — Chrome has no window names; the picker uses Window 1...N in tab-strip order.
Nothing happens on click	The dialog reports the reason on its own status line; a fetch error appears in the browser console.

6. Still true from earlier versions

- Drags start **only from the ↕ grip**; release over chip, grips, link or bare cell — over a link the tab lands just before that row.
- "Sort groups alphabetically" turns itself off on a real drag; untick restores the previous order exactly.
- Nothing is written to disk until you press **Save**.
- Session files are read only; everything is localhost.

Run the checks with `python test_editor_js.py` (58 checks).